
Logical Neural Networks

Ryan Riegel
IBM Research - Watson

Alexander Gray
IBM Research - Watson

Francois Luus
IBM Research - Africa

Naweed Khan
IBM Research - Africa

Ndivhuwo Makondo
IBM Research - Africa

Ismail Yunus Akhalwaya
IBM Research - Africa

Haifeng Qian
IBM Research - Watson

Ronald Fagin
IBM Research - Almaden

Francisco Barahona
IBM Research - Watson

Udit Sharma
IBM Research - India

Shajith Ikbāl
IBM Research - India

Hima Karanam
IBM Research - India

Sumit Neelam
IBM Research - India

Ankita Likhyani
IBM Research - India

Santosh Srivastava
IBM Research - India

Abstract

We propose a novel framework seamlessly providing key properties of both neural nets (learning) and symbolic logic (knowledge and reasoning). Every neuron has a meaning as a component of a formula in a weighted real-valued logic, yielding a highly interpretable disentangled representation. Inference is omnidirectional rather than focused on predefined target variables, and corresponds to logical reasoning, including classical first-order logic theorem proving as a special case. The model is end-to-end differentiable, and learning minimizes a novel loss function capturing logical contradiction, yielding resilience to inconsistent knowledge. It also enables the open-world assumption by maintaining bounds on truth values which can have probabilistic semantics, yielding resilience to incomplete knowledge.

1 Introduction and related work

We present *Logical Neural Networks* (LNNs), a neuro-symbolic framework designed to simultaneously provide key properties of both neural nets (NNs) (*learning*) and symbolic logic (*knowledge and reasoning*) – toward direct interpretability, utilization of rich domain knowledge realistically, and the general problem-solving ability of a full theorem prover. The central idea is to create a 1-to-1 correspondence between neurons and the elements of logical formulae, using the observation that the weights of neurons can be constrained to act as, e.g. AND or OR gates. While the view of neurons as logical gates was inherent in the seminal [13], it appears to have remained relatively unexploited since; an exception is the idea of converting logical statements into NN forms [14, 9]; the best known of these is [22], where the final model’s neurons do not necessarily retain logical gate behaviors. In LNNs, no conversions are needed because they are identical. Inputs include a propositional or first-order logic (FOL) knowledge base (KB), including the usual training data (feature-value pairs) as a special case, and which variables should be predicted from which.

Per-neuron interoperability, via full logical expressivity. Many approaches are based on Markov random fields (MRFs), such as Markov logic networks [15], probabilistic soft logic [1], and those based on ILP/SLPs e.g. [3], where each logical clause has a weight; the clauses are atomic i.e. their

internal logical structure is not represented. Obtaining probabilities from them requires an unwieldy satisfiability problem to be solved, e.g. via MCMC in [15]. LNN inference is deterministic/repeatable and provably convergent in finite steps. In LNNs, every neuron represents an element in a clause, and is either a concept (e.g. "cat") or a logical connective (e.g. AND, OR), with weights on the connecting edges. Thus each neuron represents 1) a *meaning*, raising the level of interpretability versus previous approaches, 2) a way to identify the importance of relationships between variables and 3) more parameters, defining a richer model space for potentially more accurate prediction. The network structure is thus *compositional* and *modular*, e.g. able to represent that one clause may be a sub-clause of another. The representation is *disentangled*, versus approaches such as [19, 17] that use a vector representation, sacrificing interpretability of the network. Where many approaches only allow the representational power of propositional logic or Horn clauses, LNNs allow full function-free *first-order logic* with *real values* $0 \leq x \leq 1$, and classical 0/1 logic as a special case.

Tolerance to incomplete knowledge, via truth bounds. The line of approach embodied by MRFs make a closed-world assumption, i.e. that if a statement doesn't appear in the KB, it is false. LNN does not require complete specification of all variables' exact degree of truth, more generally maintaining upper and lower bounds for each variable – allowing the *open-world assumption* that complete knowledge of the world is not realistic in general. Bounds also contain more interpretable information than single values, and we show that they can represent *probabilistic semantics*.

Many-task generality, via omnidirectional inference. LNN neurons express bidirectional relationships with each neighbor, allowing inference in any direction. This allows task generality versus typical single-task NNs, and allows full-fledged *theorem proving*. MRF approaches that hide the internal logical structure of clauses cannot draw the same conclusions that a theorem prover can. Many/most neuro-symbolic approaches, e.g. those based on embeddings, are arguably only "logic-like" and typically do not demonstrate reliably precise deduction. We show evidence of such capability on sample tasks used to test state-of-the-art theorem provers.

2 Overview

A logical neural network (LNN) is a form of recurrent neural network with a 1-to-1 correspondence to a set of logical formulae in any of various systems of *weighted, real-valued logic*, in which evaluation performs logical inference. Key innovations that set LNNs aside from other neural networks are 1) neural activation functions **constrained** to implement the truth functions of the logical operations they represent, i.e. $\wedge, \vee, \neg, \rightarrow$, and, in FOL, $\forall$ and $\exists$, 2) results expressed in terms of **bounds** on truth values so as to distinguish known, approximately known, unknown, and contradictory states, and 3) **bidirectional inference** permitting, e.g., $x \rightarrow y$ to be evaluated as usual in addition to being able to prove y given x or, just as well, $\neg x$ given $\neg y$. The nature of the modeled system of logic depends on the family of activation functions chosen for the network's neurons, which implement the logic's various atoms and operations. In particular, it is possible to constrain the network to behave exactly classically when provided classical input. Computation is characterized by tightening *bounds* on truth values at neurons pertaining to subformulae in *upward* and *downward passes* over the represented formulae's syntax trees. Bounds tightening is monotonic; accordingly, computation cannot oscillate and necessarily converges for propositional logic. Because of the network's modular construction, it is possible to partition and/or compose networks, inject formulae serving as logical constraints or queries, and control which parts of the network (or individual neurons) are trained or evaluated.

Inputs are initial truth value bounds for each of the neurons in the network; in particular, neurons pertaining to predicate atoms may be populated with truth values taken from KB data. Additional inputs may take the form of injected formulae representing a query or specific inference problem. Outputs are typically the final computed truth value bounds at one or more neurons pertaining to specific atoms or formulae of interest. In other problem contexts, the outputs of interest may instead be the neural parameters themselves — serving as a form of inductive logic programming (ILP) — after learning with a given loss function and input training data set.

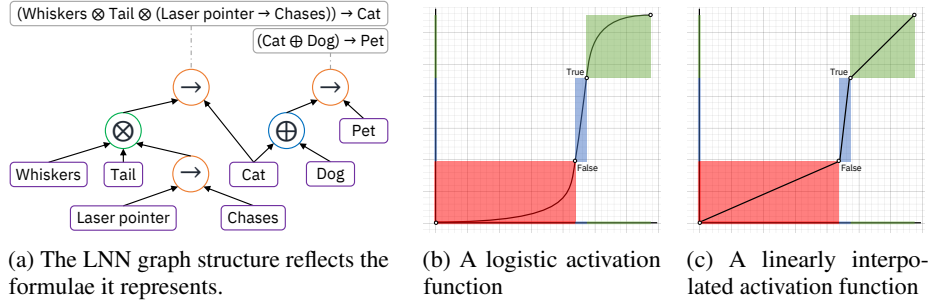


Figure 1: Neurons (1a) with alternative activation functions (1b, 1c) configured to match the truth functions of their corresponding operations, with established regions of unambiguously True, unambiguously False, and intermediate truth.

3 Model structure

In general, LNNs are described in terms of FOL, but it is useful to discuss LNNs restricted to the scope of propositional logic.¹ Structurally, an LNN is a graph made up of the syntax trees of all represented formulae connected to each other via neurons added for each proposition. Specifically, as shown in Figure 1a, there exists one neuron for each logical operation occurring in each formula and, in addition, one neuron for each unique proposition occurring in any formula. All neurons return pairs of values in the range $[0, 1]$ representing lower and upper bounds on the truth values of their corresponding subformulae and propositions. To aid interpretability of bounds, we define a threshold of truth $\frac{1}{2} < \alpha \leq 1$ such that a continuous truth value is considered True if it is greater than α and False if it is less than $1 - \alpha$. Bound values identify one of four primary states that a neuron can be in, whereas secondary states offer a more-true-than-not or more-false-than-not interpretation.

Table 1: Primary truth value bound states

Bounds	Unknown	True	False	Contradiction
Upper	$[\alpha, 1]$	$[\alpha, 1]$	$[0, 1 - \alpha]$	Lower > Upper
Lower	$[0, 1 - \alpha]$	$[\alpha, 1]$	$[0, 1 - \alpha]$	

Neurons corresponding to logical connectives accept as input the output of neurons corresponding to their operands and have activation functions configured to match the connectives' truth functions. Neurons corresponding to propositions accept as input the output of neurons established as *proofs* of bounds on the propositions' truth values and have activation functions configured to aggregate the tightest such bounds. Proofs for propositions may be established explicitly, e.g. as the heads of Horn clauses, though Section 4 shows how bidirectional inference permits every occurrence of each proposition in each formula to be used as a potential proof. Negation is modeled as a pass-through node with no parameters, canonically performing $\neg x = 1 - x$.

3.1 Activation functions for connectives

Many candidate neural activation functions can accommodate the classical truth functions of logical connectives, each varying in how it handles inputs strictly between 0 and 1. For instance, $\min\{x, y\}$ is a suitable activation function for real-valued conjunction $x \otimes y$, but so are $x \cdot y$ and $\max\{0, x + y - 1\}$.

¹First-order LNN expands to include neurons for predicate and quantifier symbols, with each formula grounding treated as a proposition. Each neuron keeps a table that maps a set of n -dimensional grounding tuples to truth value bounds, where n is the number of unique variables in the underlying subformula. Quantifiers $\forall$ and $\exists$ are modeled as pass-through nodes aggregating bounds over one of the n dimensions via min and max, respectively. Inference proceeds as described for propositional LNN in Section 4, with each grounding treated independently and special handling for $\forall$ and $\exists$. This method is similar to approaches that reduce inference in classical FOL to propositional logic [18]. Additional details are given in section A.1 of the supplementary material.